



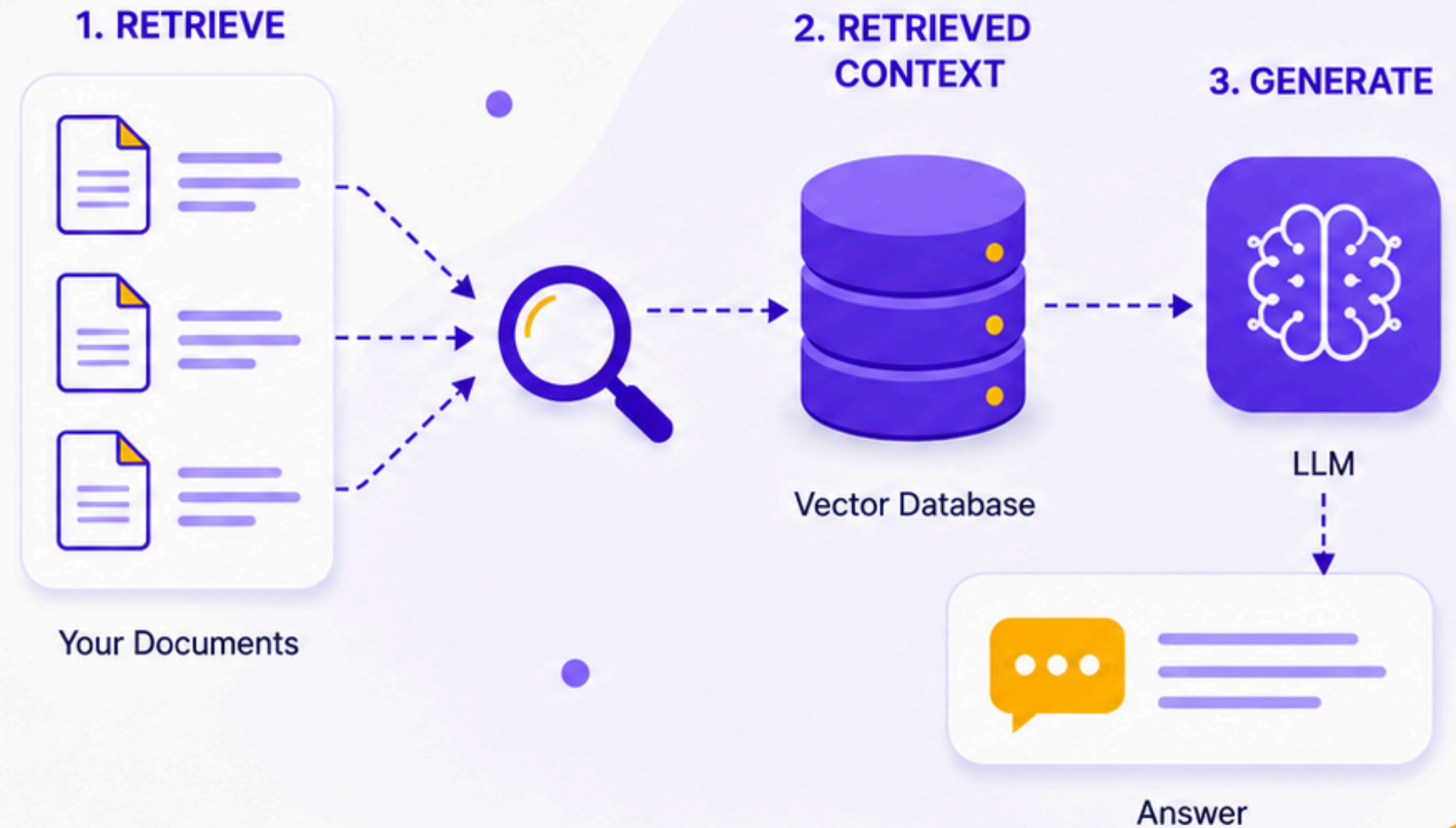
RAG

Retrieval-Augmented Generation

How AI searches documents
before answering



Presented By:
Ilyas Ait Bamou



PLAN

Presentation Plan

Here is the roadmap of our presentation about **RAG**.



01



Introduction & The Problem

02



What is RAG?

03



RAG Pipeline Overview

04



Ingestion: Text Extraction

05



Ingestion: Chunking

06



Ingestion: Embedding

07



Ingestion: Vector DB

08



Query / Answering: User Query & Query Embedding

09



Query / Answering: Similarity Search & Retrieved Context

10



Query / Answering: Augmented Prompt & Answer Generation

11



Benefits of RAG

12



Limitations of RAG

13



Technical Concepts & Definitions

14



Considerations

15



Conclusion

Introduction & The Problem

What are LLMs?

LLMs (Large Language Models) power tools like ChatGPT, Groq, and Claude. They are trained on massive amounts of data and can answer questions, write code, summarize text, and more.



But there's a catch...

The Problem: LLMs are frozen in time

An LLM is trained once and doesn't automatically update. It may not know recent events or your private documents.



1 Outdated knowledge

Recent events or new information may be unknown.



2 Hallucination

It may guess and sound confident while being wrong.



3 No private data

It never saw your contracts, reports, or internal documents.



Demo: Let's test it



Question:

What was the result of the Morocco vs Brazil match in the 2026 World Cup?



Without context:

The AI may say it doesn't know or give an unreliable answer.



With context:

The AI can answer correctly when relevant information is provided.

RAG (Retrieval-Augmented Generation)



RAG is a technique that **retrieves relevant information** from your own documents (PDFs, databases, etc.) and **injects it as context** into the LLM prompt, so the AI generates answers based on **your real data** — not just its training memory.



1. Retrieval

The system searches your documents and retrieves the most relevant information.



2. Augmentation

The retrieved information is added to the user's question as context in the prompt.



3. Generation

The LLM uses the augmented context to generate an answer based on your data.



4. Accurate Answer

The AI provides an answer grounded in your real documents, not just its training memory.



In short:

RAG = Search your data → Add it to the prompt → Get accurate, reliable answers.

RAG Pipeline Overview



1. Indexing / Ingestion



Documents / PDFs



Text Extraction



Chunking



Embeddings



Vector Database

Prepare documents so they can be searched by meaning.



2. Query / Answering



User Question



Query Embedding



Similarity Search



Retrieved Chunks



Augmented Prompt



LLM



Final Answer

When a user asks a question, the system retrieves relevant context before generating the answer.



In short:

Index data first



retrieve relevant context



generate grounded answers.



Ingestion: Text Extraction

Preparing raw documents by converting messy sources into clean, searchable text.

WHAT IS TEXT EXTRACTION?

Definition: The process of retrieving raw text from files and converting it into a clean digital format.

Input: PDFs, Word documents, scanned images, and web pages.

Output: A unified stream of plain text ready for machine processing.

WHY DO WE NEED IT?

- **Prevents Layout Chaos:** Straightens out complex multi-column layouts and sidebars.
- **Preserves Table Data:** Converts complex charts and tables into clean, structured text.
- **Eliminates Noise:** Strips out useless headers, footers, page numbers, and ads.
- **Enables Searchability:** Turns unsearchable pixels and images into actual digital words.

HOW DOES IT WORK? (THE METHODS)

Programmatic Parsers (Rule-Based)

Reads the digital text layer directly (e.g., PyPDF, BeautifulSoup).

OCR Engines (Optical Character Recognition)

Converts pixels from scans and photos into text (e.g., Tesseract).

Vision AI & IDP (Intelligent Document Processing)

Understands complex layouts, charts, and tables using advanced AI models (e.g., LlamaParse).

Ingestion: Chunking

Breaking large texts into smart pieces for better search, understanding, and performance.

WHAT IS CHUNKING?

Definition: The process of breaking down long, continuous text into smaller, meaningful segments.

The Goal:

To fit text within AI memory limits while preserving the context of the data.

The Outcome:

Smaller text pieces that are easy for computers to search, index, and understand.

WHY DO WE NEED IT?

- **Respects Model Limits:** Keeps text within the strict token boundaries of Large Language Models (LLMs).
- **Improves Search Accuracy:** Allows the vector database to pinpoint precise answers instead of scanning entire books.
- **Saves Computational Cost:** Processing smaller, highly relevant text blocks reduces AI token usage and lowers bills.
- **Boosts LLM Focus:** Prevents the AI from getting confused or “lost in the middle” of massive documents.

HOW DOES IT WORK? (THE METHODS)

Fixed-Size Chunking

Splits text by a strict count of words or characters (e.g., exactly 200 words per chunk).

Recursive/Structural Chunking

Uses formatting cues like paragraphs and punctuation to split text without cutting sentences in half.

Semantic Chunking

Uses AI to analyze meaning, drawing a boundary only when the topic changes.

Advanced / Hierarchical Chunking

Links tiny search chunks (“Child”) back to larger context chunks (“Parent”) for better retrieval.

Ingestion: Embedding

Converting text chunks into vectors so the system can understand meaning, similarity, and context.

WHAT IS AN EMBEDDING?

Definition: The process of converting text chunks into lists of numbers (vectors) that capture mathematical meaning.

The Concept: Words or phrases with similar meanings are placed close together in a multi-dimensional mathematical space.

The Outcome: Computer-readable data where the system understands concepts, synonyms, and context, not just exact keywords.

WHY DO WE NEED IT?

- **Enables Semantic Search:** Allows the AI to find answers based on meaning rather than exact keyword matching (e.g., searching for "feline" finds results about "cats").
- **Bridges Text and Math:** Converts human language into a numerical format that machine learning algorithms can calculate.
- **Powers Fast Retrieval:** Enables the database to instantly calculate the mathematical distance between a user's question and the stored data chunks.

HOW DOES IT WORK? (THE EMBEDDING METHODS)

Dense Embeddings (Semantic)

Captures deep conceptual meaning, context, and synonyms using deep learning models (e.g., OpenAI, Cohere, Hugging Face).

Sparse Embeddings (Keyword)

Focuses on exact word matching and word frequency across documents using traditional statistical algorithms (e.g., BM25, TF-IDF).

Hybrid Embeddings (Combined)

Merges both Dense and Sparse methods to find data using both conceptual meaning and exact technical keywords simultaneously.

Multimodal Embeddings

Advanced models that map both text and images into the exact same mathematical space (e.g., CLIP).

Ingestion: Vector DB

Storing and searching vector embeddings for fast, scalable, and intelligent retrieval.

WHAT IS A VECTOR DB?

Definition: A specialized database designed to store, organize, and search high-dimensional vector embeddings.

The Core Function: Unlike traditional databases that search for exact text matches, a vector database searches for mathematical similarity.

The Outcome: A highly scalable storage system built specifically for fast AI data retrieval.

WHY DO WE NEED IT?

- **Ultra-Fast Search:** Instantly compares a user's question against millions of stored chunks using advanced indexing math.
- **Maintains AI Long-Term Memory:** Acts as an external hard drive for the LLM, storing company data that the AI wasn't originally trained on.
- **Handles Scale:** Manages massive volumes of vector data efficiently without slowing down application performance.
- **Dynamic Updates:** Allows you to instantly add, update, or delete text chunks in real-time without retraining the AI model.

HOW DOES IT WORK? (THE VECTOR INDEXING METHODS)

- 1 Flat Indexing (Exact)**
Manually checks every single vector to find the perfect match; 100% accurate but very slow on big data.
- 2 HNSW (Hierarchical Navigable Small World)**
Creates a multi-layered graph network to hop between data points; the industry standard for ultra-fast, approximate searches.
- 3 IVF (Inverted File Indexing)**
Clusters similar vectors together into groups and only searches the most relevant clusters to save time.
- 4 Quantization (Compression)**
Compresses the size of the vectors to save memory space and speed up math calculations.

Query / Answering: User Query & Query Embedding

Turning a user question into a searchable query representation.



1. User Query

What

The process starts when the user asks a question in natural language.

Why

- Expresses the information need
- Guides the retrieval step
- Defines what context to search for

How

The user simply asks a question about the available knowledge or documents.



2. Query Embedding

What

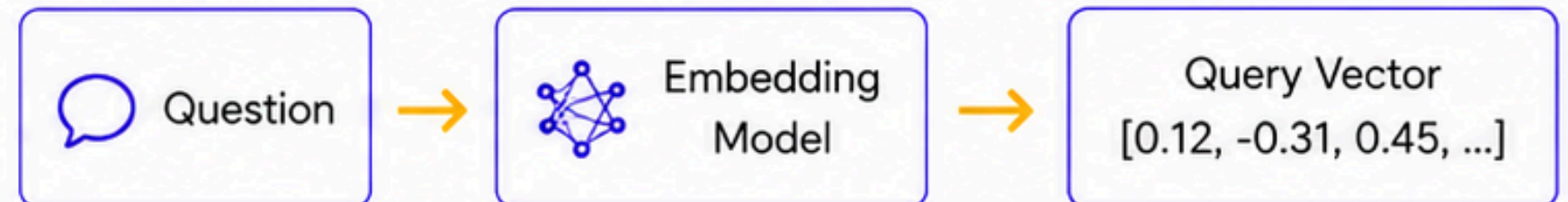
The user question is converted into a numerical vector called an embedding.

Why

- Captures semantic meaning
- Enables search by meaning, not only keywords
- Makes comparison with stored vectors possible

How

An embedding model transforms the question into a list of numbers that represents its meaning.





Query / Answering: Similarity Search & Retrieved Context



Finding the most relevant information from the vector database.



3. Similarity Search

What

The system compares the query embedding with the stored vectors in the vector database to find the most similar chunks.

Why

- Searches by meaning, not only keywords
- Finds the most relevant chunks
- Ranks results by similarity

How

The query vector is matched against stored vectors, and the nearest results are selected as top matches.



4. Retrieved Context

What

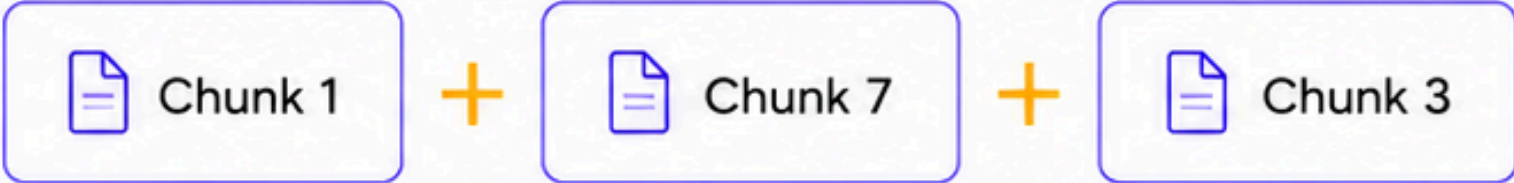
The most relevant chunks are returned and used as context for the next step in the RAG pipeline.

Why

- Provides factual grounding
- Reduces hallucination
- Prepares useful context for the prompt

How

The top matching chunks are collected and passed forward so they can be inserted into the augmented prompt.



Relevant context returned

Query / Answering: Augmented Prompt & Answer Generation

Using retrieved context to build the prompt and generate the final grounded answer.



5. Augmented Prompt

What

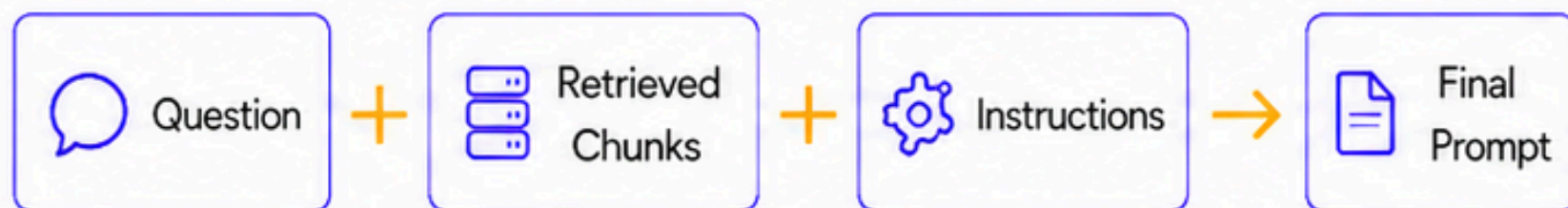
The retrieved chunks are combined with the user question and instructions to create the final prompt sent to the LLM.

Why

- Provides grounded context
- Reduces hallucination
- Gives clear instructions for the answer

How

The system inserts the user question, the top retrieved chunks, and optional rules into one structured prompt.



6. Answer Generation

What

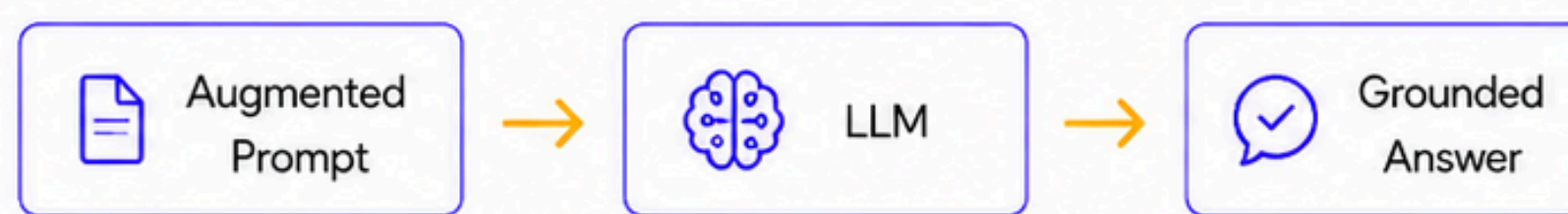
The LLM reads the augmented prompt and generates a response based on the retrieved context.

Why

- Produces a natural language answer
- Uses document-based evidence
- Returns a relevant and useful response

How

The model processes the prompt, reasons over the context, and outputs a final answer grounded in the retrieved information.



Benefits of RAG

Why retrieval-augmented generation improves accuracy, freshness, security, and trust.

ACCURACY

- **Eliminates Hallucinations:**

Forces the AI to answer using your real, verified documents instead of guessing or making things up.

- **Real-Time Data Updates:**

Connects the AI to changing data sources like manuals or market reports without expensive model retraining.

SECURITY

- **Data Privacy & Security:**

Keeps sensitive company data safely inside a private vector database; the LLM reads it only when needed.

- **Cost-Efficient Intelligence:**

Gets strong results from affordable AI models by giving them the exact context they need instead of training custom models.

TRUST

- **Source Transparency:**

Lets the AI provide citations and links back to the original documents, so humans can easily verify the answer.

Result:

more reliable, explainable, and business-ready AI answers.

Limitations of RAG

Why retrieval-augmented generation still has trade-offs in data quality, complexity, and speed.

DATA QUALITY

- **“Garbage In, Garbage Out” Dependency:**

If the initial text extraction or source data is messy, incomplete, or inaccurate, the AI’s answers will also be flawed.

- **Retrieval Failures:**

The system may fetch the wrong text chunks, causing the AI to miss the correct answer.

CONTEXT

- **Context Window Constraints:**

LLMs still have strict limits on how many chunks they can read at one time.

- **High Operational Complexity:**

Managing parsers, embedding models, vector databases, and LLMs introduces many moving parts and possible points of failure.

PERFORMANCE

- **Latency Overhead:**

The multi-step process of retrieval, prompt construction, and generation can noticeably increase response time.

Result:

RAG is powerful, but it requires clean data, strong retrieval quality, and careful system design.



Technical Concepts & Definitions

Key terms used in retrieval-augmented generation and semantic search systems.

RAG

An architectural pattern that improves AI answers by retrieving relevant external knowledge before generating a response.

LLM

The core AI model that understands text and generates natural-language answers.

Chunking

Breaking large documents into smaller, readable segments for search and processing.

Embedding

Converting text chunks into numerical vectors that capture semantic meaning.

Vector DB

A specialized database that stores embeddings and performs fast similarity search.

Hallucination

An AI error where the model confidently produces false, inaccurate, or invented information.

Relevant Data

The specific chunks that best answer a question based on semantic similarity.

The Retriever

The search component that scans the vector database and fetches the most relevant chunks.

Reranking

A second-pass filtering step that reorders retrieved chunks so the best answers appear first.

Grounding

Basing the AI's final answer directly on retrieved facts to improve truthfulness and reduce hallucinations.

Considerations

Key considerations for modern RAG systems: where data comes from and how retrieval works.

DATA SOURCES

Web & Live APIs

Fetches real-time internet data, live market feeds, and website content on the fly.

Structured Databases

Queries live SQL databases, user graphs, and CRM records alongside raw text.

Multimedia & Code

Ingests audio transcripts, software code repositories, and technical logs.

SEARCH TYPES

Keyword-Based Search (Lexical)

Matches exact words and character strings using statistical frequency (e.g., BM25). Best for exact codes, product serial numbers, and strict term matching.

Vector-Based Search (Semantic)

Matches conceptual meaning and context using embeddings and mathematical similarity. Best for understanding intent, synonyms, and related ideas.

HYBRID SEARCH

Hybrid Search (Modern Standard)

Combines keyword search and vector search to get the most accurate results.

Why it matters:

Modern RAG systems often combine multiple data sources with hybrid retrieval for stronger, more reliable answers.

Conclusion

RAG improves AI by connecting language models to relevant external knowledge before generating an answer.

HOW RAG WORKS



Retrieves relevant information from external data sources.



Adds the retrieved context to the prompt.



Generates a grounded answer using both the question and the context.

WHY IT MATTERS



Reduces hallucinations and improves answer accuracy.



Uses fresh, domain-specific knowledge without retraining the model.



Makes answers more useful, explainable, and trustworthy.

KEY TAKEAWAYS



Clean data and good retrieval quality are essential.



Chunking, embeddings, and vector databases are the backbone of RAG.



The best results come from combining strong retrieval with a capable LLM.



Final Message:

RAG turns a general LLM into a smarter, more reliable assistant connected to your own knowledge.